Riassunto - Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO, a.a. 2025/2026, Prof. Alessandro Ricci

Modulo 2.1 Asynchronous Programming (v1.3.2, 72 slide) | **Modulo 2.2** Reactive Programming (v1.2.1, 37 slide)

Riassunto basato esclusivamente sul contenuto delle slide. I numeri tra parentesi (sl. N) rimandano alla slide del modulo.

PARTE 1 - MODULO 2.1: ASYNCHRONOUS PROGRAMMING

1. Introduzione e panorama attuale (sl. 2-3)

- Stile di programmazione sempre più importante: riguarda l'esecuzione e la gestione di computazioni/richieste/processi asincroni, **astraendo dai thread**.
- Supportato da tutti i principali framework e piattaforme (Java, .NET, iOS, Android), con vari meccanismi e architetture, tra cui la programmazione event-driven ma non solo.
- È un tema ancora in evoluzione. Esempio Microsoft: prima l'Asynchronous Programming Model (APM), deprecato in favore dell'Event-based Asynchronous Pattern (EAP) con .NET 2.0, a sua volta deprecato in favore del Task-based Asynchronous Pattern (TAP) con .NET 4.5.
- **Panorama attuale**: (1) event-driven programming (architetture a event loop, reactor pattern); (2) async functions e Continuation Passing Style (event handler come continuation, supporto delle librerie di promises); (3) meccanismo async/await (che imita la programmazione sincrona pur essendo asincrono); (4) concorrenza basata su coroutine (lightweight threads).

2. Event-driven programming (sl. 4)

- Un programma è fatto di routine (**event handler**) chiamate (eseguite, attivate) quando si verificano eventi.
  - Gli eventi sono qualunque cambiamento di stato o input (asincrono) dall'ambiente rilevante per il programma: azioni utente (click, tasti), output di sensori, messaggi, risposte ricevute...
  - Il flusso di controllo dipende dagli eventi, quindi c'è **non-determinismo**.
  - "Programming without a call stack": dopo ogni esecuzione di un handler lo stack è vuoto.
- Aspetti da considerare: come gli handler sono associati agli eventi; come gli eventi sono ricevuti e gestiti; come gli handler sono eseguiti.

3. Eventi vs thread: un vecchio dibattito (sl. 5)

- Dibattito presente in diversi lavori citati nella slide: Ousterhout, "Why threads are a bad idea (for most purposes)" (USENIX 1996); Edward Lee, "The Problem with Threads" (IEEE Computer, 2006); von Behren, Condit e Brewer, "Why events are a bad idea (for high-concurrency servers)" (2003); Adya et al., Microsoft Research (2002), "Cooperative Task Management without Manual Stack Management - or, Event-driven Programming is Not the Opposite of Threaded Programming".
- Ogni approccio ha pro e contro: **la vera sfida è come metterli insieme**.

4. Modello di esecuzione: event loop (sl. 6, 8)

- Architettura di controllo a event loop (modello astratto): un **singolo thread di controllo** che attende eventi ed esegue l'handler; una **event queue** tiene traccia degli eventi generati dall'ambiente o dagli handler stessi.
- Gli event handler sono eseguiti **atomicamente**: se un evento arriva mentre un handler è in esecuzione, il suo handler viene eseguito solo dopo il completamento di quello corrente (nessuna concorrenza).
- Concettualmente un event loop può essere l'architettura di controllo di un singolo componente attivo (comportamento event-driven, coda eventi privata, task asincroni non bloccanti). Un sistema può essere composto da **più componenti attivi, ciascuno con il proprio event loop**, che non condividono memoria e

interagiscono generando eventi (sl. 8).

```
loop {
  Event ev = waitForEvent(eventQueue)
  Handler handler = selectHandler(ev)
  execute(handler)
}
```

5. Reactor pattern (sl. 7)

- La slide collega l'event loop al **Reactor pattern**, riportando estratti dal lavoro di Schmidt et al. (Pattern-Oriented Software Architecture Vol. 2, Wiley 2000) e di Schmidt, "An Object Behavioral Pattern for Demultiplexing and Dispatching Handles for Synchronous Events".
- Dagli estratti: quando un Handle associato a una sorgente di eventi diventa "ready" (es. una socket TCP pronta in lettura), l'Initiation Dispatcher usa gli Handle come "chiavi" per individuare e richiamare il metodo hook dell'Event Handler appropriato; gli estratti illustrano l'esempio di un logging server (Logging Acceptor / Logging Handler) e gli scenari di collaborazione, tra cui la connessione di un client.

6. Regola del never-blocking (sl. 9)

- Regola fondamentale: **gli event handler non devono mai bloccarsi** (nessuna chiamata bloccante) e **devono sempre terminare** (nessun ciclo infinito).
- Motivo: garantire la reattività. Una chiamata bloccante o un loop infinito bloccherebbe l'event loop, e gli eventi in coda non verrebbero più processati.
- Una chiamata bloccante va sostituita da una richiesta/computazione asincrona che genererà prima o poi un evento (risultato o errore). Le richieste/task asincroni sono serviti da altri thread, indipendenti dall'event loop, che interagiscono con esso inserendo eventi nella event queue.

7. Da event handler ad async function e callback (sl. 10-11)

- Problema: come eseguire handler a lunga durata senza bloccare e senza compromettere la reattività? Risposta: eseguendo **task/funzioni asincrone**; l'handler attiva il task e prosegue senza attendere risultato/errore.
- Problema chiave: come processare risultati o errori generati in modo asincrono dal task? Risposta: il **callback model**.
- Una funzione (callback) è passata come argomento (ultimo) della chiamata al task/funzione asincrona. Il callback viene chiamato dall'event loop quando (se) processa l'evento relativo al risultato o all'errore. È un handler "implicito" registrato dinamicamente sull'evento corrispondente.
- Il callback definisce così una **continuation** della computazione: **Continuation-Passing Style (CPS)**.

8. Continuation-Passing Style (sl. 12-14)

- In programmazione funzionale, il CPS è uno stile in cui il controllo è passato esplicitamente sotto forma di continuation. Introdotto da Gerald Jay Sussman e Guy L. Steele Jr. nel 1975 lavorando su Scheme.
- È un formato intermedio popolare per i compilatori di linguaggi funzionali: molti costrutti di controllo si esprimono elegantemente, ottimizzazione della tail call semplice. È programmazione "stack-less".
- Una funzione in CPS prende un argomento extra, una funzione di un argomento (la continuation). Quando ha calcolato il risultato lo "restituisce" chiamando la continuation con quel valore: lo stack in realtà non è usato. Il chiamante deve fornire una procedura da invocare con il valore di "ritorno".
- I callback sono come continuation chiamate quando il risultato della computazione asincrona è pronto (es. arrivo della risposta di una richiesta asincrona a un servizio remoto, risultato di un task di I/O).

```
void sum(x, y, cont){ cont(x + y); }
sum(1, 3, (res) -> { print(res) });

// versione sincrona                    // versione asincrona (findUserById e loadPic async)
function loadUserPic(userId) {          function loadUserPic(userId, ret) {
  let user = findUserById(userId);        findUserById(userId, (user) => {
  return loadPic(user.picId);               loadPic(user.picId, ret);
}                                         });
                                        }
                                        loadUserPic('john', (pic) => { ui.show(pic); });
```

9. Closure come callback (sl. 15)

- I callback sono spesso implementati come **closure**: tecnica per implementare il name binding lessicale in un linguaggio con funzioni di prima classe.
- Operativamente, una closure è un record che memorizza una funzione insieme a un ambiente: una mappa che associa ogni variabile libera della funzione al valore o riferimento a cui il nome era legato alla creazione della closure.

10. Chi invoca la callback (sl. 16-17)

- Domanda: chi invoca la continuation/callback? Due possibilità: (1) un thread di controllo diverso da quello che ha attivato la richiesta (es. il thread che genera l'evento): **problema: race**; (2) nelle architetture event-driven a event loop, la continuation è eseguita dall'event loop, cioè dal thread logico di controllo che ha attivato il task asincrono (diverso dal thread che ha servito/eseguito il task). È il modello di esecuzione delle moderne web app (client/server).
- In questo caso gli eventi sono impliciti e si riferiscono al completamento (con successo o errore) di una richiesta asincrona; gli event handler sono le continuation specificate all'invocazione; le closure definiscono tipicamente il contesto usato nel processare l'evento.
- Esempi concreti: lato client JavaScript e DART; lato server node.js, vert.x e qualsiasi framework basato su reactor.

11. Esempi in JavaScript e Node.js (sl. 18-20)

- **Client JavaScript**: l'event loop governa l'esecuzione degli script; gli eventi riguardano ciclo di vita della pagina, DOM, input utente, interazioni di rete, ecc. (come definiti dallo standard HTML5). Esempio di I/O non bloccante con `XMLHttpRequest`: si registra `onreadystatechange` e lì si definisce cosa fare dopo.
- **Modo sbagliato**: un `while` che attende `readyState === UNSENT` porta a **deadlock**, perché blocca l'event loop.
- `setTimeout(callback, timeout)` è un'altra funzione asincrona: registra un callback da eseguire allo scadere del timeout. La slide propone un esercizio: qual è l'output di un `for` con `console.log("setting timer..." + i)` e `setTimeout(() => console.log(i), 0)` (la slide non riporta la soluzione).
- **Node.js**: esempio di web server (`http.createServer`) che verifica l'esistenza di un file con `path.exists` e lo legge con `fs.readFile`, rispondendo 200 con i dati o 404; tutto con callback annidati e server in ascolto sulla porta 8080.

```
let ajaxRequest = new XMLHttpRequest;
ajaxRequest.open('GET', url);
ajaxRequest.send(null);
ajaxRequest.onreadystatechange = function() { /* what to do next */ }
```

12. Programmazione asincrona basata su continuation: punti chiave (sl. 21)

- Un **singolo thread** coordina e gestisce più task asincroni, eseguiti poi da thread esterni/supportati dal SO.
- Basso footprint di memoria rispetto alle soluzioni multi-thread (dove serve uno stack per ogni thread); si basa su un event loop implicito.
- **Nessuna race di basso livello**: nessuno stato condiviso, un solo thread accede allo stato.
- **Nessun deadlock di basso livello**: gli handler asincroni non si bloccano mai.

13. Pattern di controllo asincrono (sl. 22)

- **Sequenza** (task1 ; task2): `asyncTask1(..., (res1) -> { asyncTask2(..., (res2) -> {...}) })`, cioè il secondo task è avviato nel callback del primo.
- **Parallelo** (task1 | task2): si avviano `asyncTask1` e `asyncTask2` ciascuno con il proprio callback.

14. Problemi: callback hell (sl. 23-25)

- Oltre ai benefici, CPS e programmazione event-driven soffrono di problemi noti, detti **callback hell**:
  - **Asynchronous spaghetti**: la computazione complessiva è frammentata in handler asincroni; ciò danneggia modularità e struttura, rendendo i programmi difficili da capire e mantenere.
  - **Pyramid of doom**: la composizione in CPS porta a callback/continuation annidati, aumentando la complessità e peggiorando leggibilità, riusabilità ed estendibilità del codice.
- Esempio `step1(function(result1){ step2(function(result2){ step3(...) }) })`, con la citazione di uno sviluppatore dal Node.js Google Group: "I love async, but I can't code like this". La slide 25 ripropone gli esempi precedenti (`loadUserPic` e il server Node.js) come casi di piramide.

15. Promises (sl. 26-28)

- Il callback hell può essere **parzialmente** risolto con il meccanismo delle promise, proposto nel 1976 da Daniel Friedman e D. Wise come oggetto proxy che rappresenta un risultato ancora sconosciuto, da calcolare (simile ai future).
- Le promise rappresentano il completamento finale e il risultato di una **singola** operazione asincrona: incapsulano azioni asincrone, come un valore restituito da una computazione ma che può non essere ancora disponibile. Una promise può essere rejected o resolved **una sola volta** (immutabili una volta risolte).
- Proprietà chiave: permettono di **appiattire (flattening) l'annidamento** dei callback.
- In JavaScript sono definite nello standard (ECMA-262, 12ª edizione, giugno 2021) e dette anche **thenables**: con `then` si allegano callback da eseguire quando la promise è soddisfatta o viene sollevata un'eccezione. Supportate da diversi framework con semantiche talvolta diverse: Q (la più completa e matura), When.js (leggera), jQuery Promises (non pienamente conforme).
- Caso generale: un'operazione lunga restituisce subito un oggetto Promise e completa il lavoro in background. Si configura con `myPromise.then(onComplete, onError)`; `onError` è opzionale.

```
let promisedPic = loadUserPic('john');
promisedPic.then((pic) => { ui.show(pic); });
```

16. L'oggetto Promise e la Promise API (sl. 29-31)

- Tre stati: **pending** (appena creata), **resolved/fulfilled** (il task termina correttamente), **rejected** (il task termina con errore). Una promise non pending è detta **settled**. Una volta settled, stato e valore non cambiano più.
- Creazione: `new Promise((resolve, reject) => {...})` con `resolve(...)` in caso di successo e `reject(Error(...))` in caso di fallimento. Si allega il callback con `promise.then(onSuccess, onError)`.
- Esempio `delayWithRandom(t, p)`: funzione asincrona che restituisce una Promise; dopo `t` ms un `setTimeout` genera un numero casuale `r` e chiama `resolve(r)` se `r > p`, altrimenti `reject(r)`. Il chiamante usa `.then` con un handler di successo e uno di fallimento.

17. Promise chaining e propagazione dei valori (sl. 32-37)

- **Chaining**: caratteristica chiave per appiattire la piramide. `then` restituisce a sua volta una promise, quindi le promise possono essere concatenate (la slide mostra due forme equivalenti: con variabile intermedia `p1` oppure con `.then(...).then(...)` in cascata; l'esempio stampa 1 e poi 3).
- **Propagazione**: il valore di ogni promise nella catena è quello restituito dal precedente `onComplete/onError`.

- **Concatenare funzioni asincrone** (non solo la prima che genera la promise): si restituiscono promise dentro la catena (esempio con `delayWithRand` richiamata con 1000, 1500 e 500 ms).
- **Pattern di flusso sequenziale** reso più semplice: `asyncFunc1(...).then(v => { ...; return asyncFunc2(...) }).then(v => { ...; return asyncFunc3(...) })`.
- **Flattening della piramide**: `findUserById('john').then(user => findPic(user.picId)).then(pic => ui.show(pic))` al posto delle chiamate annidate.
- **Pattern di flusso parallelo**: più `then` sulla stessa promise (`myPromise`); entrambi i callback vengono chiamati al termine di `asyncFunc1`, con lo stesso valore contenuto nella promise.

18. Promisifying existing code (sl. 38)

- Esempio (da html5rocks.com) di funzione `get(url)` che incapsula una richiesta XHR in una nuova Promise: in `onload` controlla lo status (anche 404 arriva a `onload`); se 200 fa `resolve(req.response)`, altrimenti `reject(Error(req.statusText))`; in `onerror` fa `reject(Error("Network Error"))`; poi `req.send()`. L'uso è `get('story.json').then(success, failure)`.

19. Combinare promise: all e race (sl. 39-40)

- `Promise.all(promiseList)`: compone un insieme di chiamate asincrone indipendenti da eseguire in parallelo, con un **join point**. Il callback è chiamato quando tutte le promise sono risolte o almeno una è rifiutata. Nell'esempio riceve un array `values` con i risultati.
- `Promise.race(promiseList)`: compone chiamate asincrone indipendenti ma completa **appena una di esse completa**. Il callback è chiamato quando una promise è risolta o rifiutata.

20. Gestione degli errori con le promise (sl. 41)

- Miglioramento significativo rispetto ai callback. Regole:
  - Qualsiasi handler che restituisce un valore e non solleva errori passa un successo alla promise successiva, inclusi gli handler `onFailure`.
  - Qualsiasi handler che solleva un'eccezione passa un reject alla promise successiva.
  - Uno stato non gestito è passato alla promise successiva nella catena: se non c'è un reject handler e la promise è rifiutata, lo stato passa al prossimo handler; quando lo stato trova un handler il processing riprende normalmente. È simile al caso sincrono.

21. Promise immediatamente risolte, task e microtask (sl. 42-44)

- `Promise.resolve(...)` costruisce promise subito risolte; con `then` si può concatenare un callback da eseguire nel **ciclo successivo** dell'event loop (esempio: stampa "this cycle" e poi "next cycle").
- In JavaScript/Web la event queue (o dispatch queue) ospita sia **task** sia **microtask**. L'event loop funziona come di consueto: prende i task uno alla volta e li lascia eseguire fino al completamento prima di servire il successivo; i task possono anche essere estratti preferenzialmente (priorità).
- I **microtask** sono una versione leggera del task; caratteristica chiave: sono garantiti essere eseguiti **prima che parta il task successivo**; la microtask queue è processata subito, appena possibile. API per aggiungerli: `Promise.resolve().then(() => {...})`.
- Figura (da dartlang.org, event loop): dopo l'esecuzione di `main()` l'event loop controlla se la microtask queue è vuota; se no esegue il prossimo microtask, altrimenti controlla la event queue e gestisce il prossimo evento (es. key, timer, click, ciascuno con il proprio handler), finché entrambe le code sono vuote e l'app può terminare. **Mentre l'event loop esegue microtask, la event queue è bloccata**: l'app non può disegnare grafica, gestire click o reagire all'I/O.

22. I problemi delle promise (sl. 45-47)

- **Eagerness e problema del parametro di** `then`: non si possono passare parametri alle funzioni usate in un `then`; le promise sono eager, cioè iniziano il lavoro appena costruite. Esempio `delay(t)` e `printTime()`: scrivere `.then(delay(1000))` non funziona come atteso (viene passata una promise, già avviata); la versione corretta usa `.then(() => { return delay(1000); })`.

- **Non utilizzabili con cicli/iterazioni**: un `for` che chiama `delay(1000).then(...)` a ogni iterazione "doesn't work as expected" (le slide non entrano oltre nel dettaglio).
- **Cancellazione**: le promise non possono essere cancellate, nessun controllo sul task asincrono, con possibile spreco di risorse.

23. Da Promise ad async/await (sl. 48-51)

- Estensione del linguaggio **async & await**: la soluzione più recente e promettente nella programmazione asincrona. Obiettivo: fornire uno **stile di programmazione sincrono mantenendo un nucleo asincrono**.
- Supporto: introdotto in ES2017 (8ª edizione della specifica JavaScript, giugno 2017); implementato nel TAP di Microsoft .NET (usabile in C#); presente in un numero crescente di linguaggi (Python, DART...); lista aggiornata su Wikipedia (Async/await).
- Idea principale: invece di `p.then(onCompleteFunc, onErrorFunc)` si scrive `let res = await p; <use res>`, cioè si attende concettualmente il risultato (**cedendo il controllo**) e si prosegue (riprendendo il controllo) quando la promise è risolta.
- `await <promise>`: applicabile a qualunque Promise, sospende l'esecuzione della funzione finché la promise è soddisfatta, **ma non blocca il flusso di controllo**: il controllo è ceduto (yield) salvando lo stato corrente della computazione.
- `await` **solo dentro funzioni** `async`: può essere usato soltanto nel corpo di funzioni etichettate `async` (funzioni dichiarate, annidate o assegnate a variabili).

24. Async functions (sl. 52-53)

- Le funzioni `async` sono funzioni **sospendibili che restituiscono una promise**. La funzione viene convertita in un oggetto `AsyncFunction`: è solo definita, non eseguita.
- Chiamandola, il codice è eseguito come una funzione normale (nello stesso ciclo) e restituisce una promise. Output dell'esempio: `pre`, `[AsyncFunction]`, `Promise {}`, `post`.
- L'oggetto `AsyncFunction` contiene il meccanismo di basso livello (basato su **coroutine**, discusse più avanti) che permette di esprimere il corpo in stile sincrono.
- Una async function può essere chiamata **con o senza** `await`, con comportamento diverso: l'esempio `pause(t)` (stampa "before", attende una promise con `setTimeout`, stampa "after") è chiamata una volta semplicemente (`pause(1000)`) e una volta con `await pause(1000)` dentro un `main` async, tra due `console.log` ("before call" / "after call").

25. Dettagli su await e coroutine (sl. 54-55)

- `await <promise>` converte una Promise in un risultato o in un'eccezione come se fosse il risultato di una funzione sincrona: se la promise è risolta estrae e restituisce il valore; se è rifiutata **lancia un'eccezione** con il valore del rifiuto.
- `await` si comporta come un **return che può essere ripreso**: all'esecuzione di un await il thread torna al programma chiamante, che riceve l'oggetto Promise creato dalla async function in stato pending; lo stato della computazione è salvato e ripristinato al ritorno del controllo; la funzione riprende in modo asincrono, appena il thread dell'event loop preleva il corrispondente evento dalla event queue.
- Per rilasciare e riprendere il controllo si sfrutta il meccanismo delle **coroutine**: vecchio meccanismo per lo scheduling cooperativo, riportato in auge nella programmazione asincrona a singolo flusso di controllo.

26. Benefici e problemi di async/await (sl. 56-60)

- **Benefici**: programmazione asincrona molto semplificata; non servono esplicitamente Promise/then/catene per il pattern sequenziale; i problemi di eagerness e del parametro di `then` sono risolti; si può usare con **cicli/iterazioni** (esempio: `for` con `await waitFor(1000)` che stampa "Step i" tra "Before" e "After").
- **Problemi**:
  - `await` solo dentro una funzione async, quindi non a livello top: serve almeno una funzione async `main`.
  - Una async function non può riprendere mentre il thread è occupato con altri handler: la semantica dell'event loop è preservata.

  - I blocchi `{...}` **non sono più garantiti atomici**: l'esecuzione di un singolo blocco può estendersi su più iterazioni dell'event loop, con possibili **race condition**.
  - La **composizione** richiede di mischiare await/async con la Promise API (esempio: `await Promise.all([f(), g()])` dove `f` attende 1000 ms e restituisce 3, `g` attende 5000 ms e restituisce 5).
- **"Design clash"** tra sincrono e asincrono: serve una forte disciplina nel mescolare i due stili per evitare programmi dal comportamento difficile da capire; ci si chiede quali concetti astratti permettano di pensare/progettare un programma al di là dei meccanismi di programmazione; dubbi su **modularità, riuso ed estendibilità**. Serve un livello di astrazioni più alto che integri naturalmente prospettiva sincrona e asincrona.
- **Modularità/incapsulamento**: nell'esempio con `delay`, `f` e `g`, se `g` non è `async` e chiama `f()` senza `await` il programma "does not work as expected"; la versione corretta rende `g` async e usa `await f()`. Cioè chi usa una funzione deve conoscere se è asincrona o meno.

27. Coroutine (sl. 61-63)

- Generalizzazione del concetto di subroutine: l'esecuzione può essere **sospesa e ripresa** [Con63]. Per multitasking cooperativo non preemptive; mattone di base per task cooperativi, eccezioni, event loop, iteratori, liste infinite e pipe. **Non è legato a eventi e programmazione event-driven.**
- Storicamente nascono come metodo in assembly, ma sono supportate da alcuni linguaggi di alto livello: primi esempi Simula, Smalltalk, Modula-2; esempi più recenti Ruby, Lua, Julia, Go, Kotlin. Rinnovata attenzione recente soprattutto per la programmazione asincrona.
- Come routine generalizzata: una coroutine può uscire chiamando altre coroutine, che possono poi tornare al punto in cui erano state invocate; dal punto di vista della coroutine non sta uscendo ma chiamandone un'altra, il che si fa con lo "**yielding**". Esempio (da Wikipedia): sistema produttore/consumatore cooperativo a un solo flusso, con coroutine `produce` e `consume` che si cedono il controllo a vicenda (yield to consume / yield to produce) lavorando su una coda `q`.
- **Caratteristiche**: un'istanza di coroutine mantiene stato e varia tra le invocazioni (possono esserci più istanze contemporanee); le subroutine tradizionali sono casi speciali di coroutine (una subroutine è una coroutine che non chiama yield); si implementano facilmente nei linguaggi che supportano nativamente le continuation; **non** si possono implementare come semplice libreria sopra un linguaggio basato solo su subroutine e meccanismi simili (es. metodi).

28. Concorrenza basata su coroutine e Kotlin (sl. 64-68)

- Le coroutine sono un meccanismo di basso livello per implementare **lightweight thread**: un thread può contenere molte coroutine ma solo una è in esecuzione in un dato momento. Sono veloci ed economiche da creare e gestire, **senza context switching del SO**.
- Esempio **Kotlin**, concetti principali: suspending computations, coroutine dispatcher, coroutine builder.
- **Suspending computations**: computazioni che possono sospendere la propria esecuzione senza bloccare il thread in cui risiedono, che può essere usato per altre computazioni finché non servono di nuovo. Tipi: **suspending function** (modificatore `suspend`) e **suspending lambda** (come una lambda normale ma può sospendersi chiamando altre suspending function).
- Esempio `greetAfter(name, delayMillis)`: l'esecuzione si sospende alla chiamata di `delay()` (a sua volta suspending, sospende per una durata); finito il delay riprende normalmente; nel frattempo il thread può fare altro.
- **Coroutine dispatcher**: decide su quale thread avviare o riprendere una coroutine. **Coroutine builder**: funzione che prende una suspending lambda e crea una coroutine per eseguirla. Builder di Kotlin: `async()` (avvia una coroutine quando ci si aspetta un risultato), `launch()` (avvia una coroutine che non restituisce risultato), `runBlocking()` (collega codice bloccante a codice sospendibile).
- **Esempio giocattolo (sl. 65)**: in `runBlocking` si lancia un `async` che stampa, fa `delay(1000)`, stampa e restituisce 100; nel frattempo si chiama `greetDelayed(200)` e infine `result.await()`. Output: before async call / after the async call, before greet / inside the async call / Hello, World! / after greet trigger / exiting the async call / 100.

29. Fibers e virtual threads (sl. 69-70)

- Una **fiber** è un thread di esecuzione leggero. Come i thread condivide lo spazio di indirizzamento; diversamente dai thread usa **multitasking cooperativo** (i thread usano multitasking preemptive affidandosi allo scheduler del kernel), cioè le fiber cedono esse stesse il controllo a un'altra fiber, quindi possono essere implementate con un solo thread.
- Fiber e coroutine sono simili ma a livelli di astrazione diversi: le coroutine sono un costrutto a livello di linguaggio (forma di flusso di controllo), le fiber un costrutto a livello di sistema (thread che non girano in parallelo). Le fiber possono essere viste come implementazione di coroutine o come substrato su cui implementarle.
- Implementate da diversi SO/framework/linguaggi (UNIX/pthreads, Windows/.NET) e in Java come **virtual threads (da JDK 19)**.
- **Virtual threads vs programmazione asincrona: dibattito senza fine.** Secondo l'overview di Brian Goetz [Goe22]: i virtual thread offrono migliore modularità e incapsulamento per modularizzare processi logici e flussi di controllo (rispetto alla programmazione asincrona); tuttavia, come i thread fisici, **non offrono un trattamento di prima classe per eventi e computazioni reattive**, che devono essere implementati e gestiti a livello applicativo.

30. Riferimenti del modulo 2.1 (sl. 71-72)

- Burnham (AsyncJavascript), Van Cutsem (Event-based Concurrency Control, OOPSLA 2009), Hohpe (Programming Without a Call Stack), MSDN (Asynchronous programming su .NET 4.5), Tilkov e Vinoski (Node.js, IEEE Internet Computing 2010), Ian Elliot (JavaScript Async), specifiche EcmaScript 6.0 e 8.0; [Con63] Conway, [Mou04] de Moura e Ierusalimschy "Revisiting Coroutines", [Pro18] Prokopec (Coroutines with Snapshots, ECOOP 2018), [Goe22] Goetz (Virtual Threads), articolo "JVM has coroutine/continuation/fiber etc.".

PARTE 2 - MODULO 2.2: REACTIVE PROGRAMMING

1. Sommario e panoramica (sl. 2-3)

- Temi del modulo: overview della reactive programming (RP) e radici di ricerca (paper di riferimento: Bainomugisha, Lombide Carreton, van Cutsem, Mostinckx, de Meuter, "A survey on reactive programming", ACM Computing Surveys 45(4), art. 52, agosto 2013); astrazioni di base (**behaviours ed eventi**); RP nel mainstream: **Reactive Extensions (Rx)**, con observable e observer.
- CPS e Promise assumono che le computazioni asincrone consegnino il risultato **in un colpo solo (one-shot)**; nelle applicazioni è invece comune dover gestire **stream asincroni di dati/eventi**: è l'obiettivo della RP (programmazione con stream di dati asincroni).
- Paradigma orientato ai **flussi di dati e alla propagazione del cambiamento**, che facilita la gestione di stream asincroni di dati ed eventi. Fortemente legato al pattern **Observer** e alla programmazione event-driven. Introdotta negli anni '90, oggi attrae molta attenzione come approccio efficace per sviluppare app (web) reattive, in particolare nel contesto dei Big data.

2. Linguaggi e framework (sl. 4)

- Estensioni reattive introdotte per tutti i principali linguaggi/sistemi: FrTime (basato su Scheme), Reactive Extensions (Rx) di Microsoft, Reactive JavaScript (Flapjax, Bacon.js, AngularJs), Reactive DART, Scala.React, RxJava...

3. Radici di ricerca (sl. 5)

- La maggior parte della ricerca sulla RP discende dal linguaggio **Fran** [Elliott e Hudak 1997]: DSL funzionale sviluppato alla fine degli anni '90 per facilitare la costruzione di applicazioni grafiche e di media interattivi.
- **Functional Reactive Programming (FRP)**: basata sul paradigma di programmazione **synchronous dataflow** [Lee e Messerschmitt 1987] ma con vincoli real-time rilassati. Proposta di recente come soluzione ai problemi e alla complessità dello sviluppo di applicazioni event-driven.

4. Problemi della programmazione asincrona (sl. 6-7)

- Programmi e applicazioni asincroni sono difficili da scrivere con approcci sequenziali convenzionali, perché è impossibile prevedere o controllare l'ordine di arrivo degli eventi esterni: il controllo salta tra gli event handler al mutare imprevisto dell'ambiente.
- **Inversion of control**: il flusso di controllo è guidato da eventi esterni e non da un ordine specificato dal programmatore. Quando cambia lo stato di una computazione o di un dato, il programmatore deve **aggiornare manualmente** tutto ciò che ne dipende; tale gestione manuale di cambiamenti di stato e dipendenze è complessa e soggetta a errori (es. cambiamenti nel momento o nell'ordine sbagliato) [Cooper e Krishnamurthi 2006].
- Coordinare i callback è difficile anche per programmatori esperti: molti frammenti di codice isolati possono manipolare gli stessi dati con ordine di esecuzione imprevedibile (**"asynchronous spaghetti"**). Poiché i callback di solito non restituiscono un valore, devono compiere **side effect** per influenzare lo stato dell'applicazione [Cooper 2008].

5. Il paradigma RP (sl. 8)

- Fornisce astrazioni per esprimere i programmi come **reazioni a eventi esterni**, lasciando al linguaggio la gestione automatica del flusso del tempo (supportando concettualmente la simultaneità) e delle dipendenze tra dati e computazioni: astrae dalla gestione del tempo come i garbage collector astraggono dalla gestione della memoria.
- Facilita lo sviluppo **dichiarativo** di applicazioni event-driven: si esprime *cosa* fare e si lascia al linguaggio la gestione di *quando* farlo. È costruita attorno a valori **continui che variano nel tempo** e alla **propagazione del cambiamento**: i cambiamenti di stato sono propagati automaticamente ed efficientemente attraverso la rete di computazioni dipendenti dal modello di esecuzione sottostante.

6. Esempio di propagazione del cambiamento (sl. 9)

- `var1 = 1; var2 = 2; var3 = var1 + var2`. In RP il valore di `var3` è sempre aggiornato: viene ricalcolato automaticamente ogni volta che `var1` o `var2` cambiano. I valori cambiano nel tempo e, quando cambiano, tutte le computazioni dipendenti sono rieseguite automaticamente. `var3` è detta **dipendente** da `var1` e `var2` (la slide mostra il grafo delle dipendenze dalla survey).

7. Astrazioni reattive (sl. 10)

- La RP astrae eventi che variano nel tempo per il consumo da parte del programmatore, che definisce elementi che reagiscono a ciascun evento in arrivo. Le astrazioni sono **valori di prima classe**: possono essere passate in giro o composte nel programma.
- Due tipi principali:
  - **Event stream**: modellano valori tempo-varianti discreti (o sparsi); astrazioni asincrone per il flusso progressivo di dati intermittenti e sequenziali provenienti da un evento ricorrente. Es.: gli eventi del mouse.
  - **Behaviour (o signal)**: valori **continui** nel tempo; astrazioni per il flusso ininterrotto e fluido di dati in arrivo da un evento stabile. Es.: un timer.

8. Esempio in Flapjax: timer (sl. 11)

```
var timer = timerB(100);
var seconds = liftB(function (time){ return Math.floor(time / 1000); }, timer);
insertDomB(seconds, 'timer-div');
```

- Si imposta un elemento DOM che mostra il conteggio del timer in secondi: prima si crea un behaviour `timer` che produce un valore ogni 100 ms; per mostrare i secondi si definisce un ulteriore behaviour `seconds`; infine `seconds` è inserito nell'elemento DOM con ID `timer-div`: il valore è quindi continuamente aggiornato e visualizzato. **Punto chiave: modo completamente dichiarativo di gestire eventi e stream di eventi.**

9. Modello di valutazione e modelli pull/push (sl. 12-13)

- Il modello di valutazione riguarda **come i cambiamenti sono propagati** lungo il grafo delle dipendenze di valori e computazioni. Dal punto di vista del programmatore la propagazione è automatica: un cambiamento è propagato a tutte le computazioni dipendenti; quando c'è un evento a una sorgente, le computazioni dipendenti sono notificate, possibilmente innescando un ricalcolo.
- Scelta di progetto a livello linguaggio: **chi inizia la propagazione?** La sorgente "spinge" (push) i nuovi dati ai dipendenti (consumatori) o i dipendenti li "tirano" (pull) dalla sorgente (produttore)? In entrambi i casi la sequenza di valori fluisce dal produttore al consumatore.

Modello	Descrizione
Pull-based	La computazione che richiede un valore lo "tira" dalla sorgente. Propagazione guidata dalla **domanda** (demand-driven). Implementabile da linguaggi lazy (es. Haskell).
Push-based	Quando la sorgente ha nuovi dati li "spinge" alle computazioni dipendenti. Propagazione guidata dalla **disponibilità** di nuovi dati (data-driven). Implementato da linguaggi eager (es. Flapjax basato su JavaScript, Scala.React).

10. Glitch e glitch avoidance (sl. 14-15)

- I **glitch** sono inconsistenze di aggiornamento che possono verificarsi durante la propagazione: se una computazione viene eseguita prima che tutte le espressioni da cui dipende siano valutate, valori nuovi possono combinarsi con valori obsoleti (stale) [Cooper e Krishnamurthi 2006]. Problema che riguarda **solo il modello push-based**.
- **Esempio**: `var1 = 1; var2 = var1 * 1; var3 = var1 + var2`. Con `var1 = 1` si ha `var2 = 1` e `var3 = 2`. Se `var1` diventa 2 ci si aspetta `var2 = 2` e `var3 = 4`; ma in un'implementazione reattiva ingenua `var1 + var2` può essere ricalcolata prima di `var1 * 1`: `var3` vale momentaneamente **3** (scorretto), poi `var2` è ricalcolata e `var3` raggiunge il valore corretto 4.

- La survey cita come soluzione la valutazione in un grafo ordinato topologicamente [Cooper e Krishnamurthi 2006].
- **Glitch avoidance**: la maggior parte delle implementazioni reattive recenti la ottiene per programmi reattivi su **un solo computer**, ma non per programmi reattivi **distribuiti**: evitare glitch in un contesto distribuito non è immediato per via di guasti di rete, ritardi e assenza di un clock globale. È un problema di ricerca aperto [Salvaneschi et al. 2014, Mogk et al. 2018].

11. Lifting (sl. 16-17)

- Le espressioni che dipendono da valori reattivi devono essere a loro volta reattive: una variabile a cui è assegnata un'espressione che coinvolge behaviour o event stream diventa essa stessa una **variabile reattiva**, perché gli aggiornamenti dei behaviour/stream la influenzano. Il processo di conversione di una variabile normale in reattiva è detto **lifting**.
- Per ricalcolare correttamente tutte le espressioni reattive quando uno stream o behaviour scatta, la maggior parte delle librerie costruisce un **grafo delle dipendenze** dietro le quinte: quando un'espressione cambia, quelle dipendenti sono ricalcolate e aggiornate. Nell'esempio del timer, un cambio di `timer` innesca la rivalutazione di `function (time){...}` e l'aggiornamento di `seconds`, che a sua volta aggiorna il valore inserito nella pagina web.
- **Lifting implicito vs esplicito**: alcune librerie JavaScript fanno il lifting implicitamente (es. Bacon.js); in altre il programmatore lo fa manualmente (es. React.js); una terza categoria offre entrambi (es. Flapjax: come libreria il lifting è esplicito, come compilatore il codice è trasformato implicitamente).

12. Composizione di event stream e behaviour (sl. 18-19)

- La **composizione di astrazioni reattive** è una caratteristica essenziale: permette di evitare il "callback nightmare". Ad esempio, invece di tre callback separati per click, movimento e rilascio del mouse, si possono comporre in un unico event stream che risponde a tutti e tre.
- La maggior parte delle librerie JavaScript offre questo supporto: in Bacon.js le properties (behaviour) si compongono con l'operatore `combine`; in Flapjax è fornita la funzione `mergeE` (la slide dice che "throttles" l'ultimo stream in arrivo da uno dei due eventi).

```
var saveTimer = timerE(10000); //10 seconds
var saveClicked = extractEventE('save-button','click');
var save = mergeE(saveTimer,saveClicked);
save.mapE(doSave); //save received data
```

- Tre event stream: `saveTimer` scatta ogni 10 secondi; `saveClicked` scatta a ogni click sul pulsante di salvataggio; `save` scatta quando scade il timer **o** si clicca il pulsante; infine per ogni elemento dello stream `save` viene chiamata `doSave` (stile funzionale).

13. Direzioni di ricerca in corso (sl. 20)

- **Distributed reactive programming** (composizione di stream reattivi distribuiti) e **gestione della fault tolerance**.

14. Librerie reattive disponibili (sl. 21)

- Integrazione della RP in linguaggi/paradigmi esistenti. Esempi principali:
  - **Reactive Extensions (Rx)**: API per la programmazione asincrona con observable stream, disponibile su più piattaforme (reactivex.io).
  - **Reactive Streams**: iniziativa per fornire uno standard per l'elaborazione asincrona di stream con **backpressure non bloccante**, per ambienti JVM e JavaScript (reactive-streams.org).
  - **Project Reactor**: libreria reattiva di quarta generazione basata sulla specifica Reactive Streams, per costruire applicazioni non bloccanti sulla JVM (projectreactor.io).

15. Caratteristiche e concetti comuni (sl. 22-28; da Project Reactor)

- Elenco: composability e readability; dati come flusso manipolato con un ricco vocabolario di operatori; **nulla accade finché non ci si sottoscrive**; **backpressure**, cioè la capacità del consumatore di segnalare al produttore che il ritmo di emissione è troppo alto.

- **Composabilità e leggibilità** (sl. 23): capacità di orchestrare più task asincroni usando i risultati dei precedenti come input dei successivi. È strettamente legata a leggibilità e manutenibilità: al crescere di numero e complessità dei livelli di processi asincroni, comporre e leggere il codice diventa sempre più difficile (cfr. "callback hell").
- **Analogia della catena di montaggio** (sl. 24): i dati elaborati da un'applicazione reattiva si muovono come su una catena di montaggio; la materia prima arriva da una sorgente (il Publisher originale) e diventa un prodotto finito da consegnare al consumatore (Subscriber); può attraversare trasformazioni e passi intermedi o far parte di una catena più grande che aggrega pezzi intermedi. Se c'è un intoppo in un punto (es. imballare richiede troppo tempo), la postazione in difficoltà può **segnalare a monte** di limitare il flusso di materia prima.
- **Operatori** (sl. 25): sono come le postazioni della catena; ciascun operatore aggiunge comportamento a un Publisher e **avvolge il Publisher del passo precedente in una nuova istanza**. L'intera catena è collegata: i dati originano dal primo Publisher e scendono lungo la catena, trasformati a ogni anello, finché un Subscriber conclude il processo. Esistono molti tipi di operatori: da semplice trasformazione e filtraggio fino a orchestrazione complessa e gestione degli errori.
- **Nothing happens until you subscribe** (sl. 26): in una catena di Publisher di solito i dati non iniziano a fluire per default. Prima si crea la descrizione astratta del processo asincrono (**fase di configurazione**); poi, sottoscrivendosi, un Publisher è legato a un Subscriber, il che innesca il flusso dei dati in tutta la catena. Internamente ciò avviene tramite un singolo segnale di **request** dal Subscriber, propagato a monte fino al Publisher sorgente.
- **Backpressure** (sl. 27): nell'analogia è un segnale di feedback inviato a monte quando una postazione elabora più lentamente di quella che la precede. Un subscriber può lavorare in modalità **unbounded** (la sorgente spinge tutto alla massima velocità possibile) oppure usare il meccanismo di **request** per segnalare di essere pronto a elaborare al massimo *n* elementi. Ciò trasforma il modello push in un **ibrido push-pull**: il downstream può "tirare" *n* elementi dall'upstream se sono prontamente disponibili; se non lo sono, vengono spinti dall'upstream appena prodotti. (La slide rimanda a approfondimenti successivi su ReactiveX.)
- **Hot vs cold** (sl. 28): la famiglia di librerie Rx distingue due categorie di reactive stream in base a come reagiscono ai sottoscrittori:

Tipo	Comportamento
Cold stream	Avvia un nuovo stream per ogni Subscriber, **anche alla sorgente dei dati**. Tipicamente ogni subscriber riceve tutti gli elementi dello stream indipendentemente dal momento in cui si sottoscrive.
Hot stream	Non riparte da zero per ogni Subscriber: i subscriber in ritardo ricevono i segnali emessi **dopo** la loro sottoscrizione. Alcuni hot stream possono mettere in cache o riprodurre (replay) la storia delle emissioni, in tutto o in parte. In generale una sequenza hot può emettere anche quando nessun subscriber è in ascolto: un'eccezione a "nothing happens before you subscribe".

16. Reactive Extensions (Rx) (sl. 29-31)

- Rx è stata originariamente introdotta come libreria su **.NET**, ispirata alla ricerca sulla FRP. Oggi esistono estensioni reattive per quasi ogni linguaggio (reactivex.io): RxJS, RxJava, RxScala, RxPython... Adottate da diversi framework web/mobile, sia frontend sia backend (es. Angular).
- Descrizione: "The Observer pattern done right"; "combinazione delle migliori idee del pattern Observer, del pattern Iterator e della programmazione funzionale". Visione principale spiegata in Erik Meijer, "Your mouse is a database", Commun. ACM 55(5), 2012.
- **Tre proprietà core**: (1) **asincrono ed event-based**: la missione di Rx è semplificare questi modelli di programmazione, aspetto chiave per interfacce utente reattive, web app reattive, applicazioni cloud dove l'asincronia è la quintessenza; (2) **composizione**: semplifica in particolare la composizione di computazioni asincrone; (3) **observable collections**: sfrutta la conoscenza di modelli come **LINQ**, guardando alle computazioni asincrone come sorgenti dati osservabili. Il mouse "diventa un database di movimenti e click": tali sorgenti sono composte con combinatori in senso LINQ (filtri, proiezioni, join, operazioni temporali,

ecc.).

- **Principi di Rx**: (1) rappresentare più stream di dati asincroni (observable stream) da fonti diverse (es. quotazioni di borsa, tweet, eventi del computer, richieste a web service); (2) sottoscriversi allo stream con l'interfaccia `IObserver<T>` (su .NET): `IObservable<T>` notifica l'`IObserver<T>` sottoscritto a ogni evento. Poiché le sequenze osservabili sono stream di dati, si possono interrogare con i **query operator standard di LINQ** (metodi di estensione di Observable): filtrare, proiettare, aggregare, comporre ed eseguire operazioni temporali su più eventi. Esistono inoltre altri operatori specifici per reactive stream; **cancellazione, eccezioni e sincronizzazione** sono gestite con i metodi di estensione di Rx.

17. IObservable in .NET (sl. 32-33)

- Rx si integra con gli stream sincroni (`IEnumerable<T>`) e con le computazioni asincrone a singolo valore (`Task<T>`), come mostra la tabella:

	Singolo valore di ritorno	Più valori di ritorno
Pull / Synchronous / Interactive	`T`	`IEnumerable<T>`
Push / Asynchronous / Reactive	`Task<T>`	`IObservable<T>`

- Interfacce: `IObservable<T>` ha `IDisposable Subscribe(IObserver<T> observer)`; `IDisposable` ha `void Dispose()`; `IObserver<T>` ha `void OnNext(T value)`, `void OnError(Exception error)`, `void OnCompleted()`.

18. Esempi semplici in .NET (sl. 34)

```
IObservable<int> source = Observable.Range(1, 10);
IDisposable subscription = source.Subscribe(
  x  => Console.WriteLine("OnNext: {0}", x),
  ex => Console.WriteLine("OnError: {0}", ex.Message),
  () => Console.WriteLine("OnCompleted"));
Console.ReadLine();
subscription.Dispose();
```

- Quando un observer si sottoscrive a una sequenza osservabile, il thread che chiama `Subscribe` può essere diverso da quello in cui la sequenza gira fino al completamento: la chiamata a `Subscribe` è **asincrona**, cioè il chiamante non è bloccato fino alla fine dell'osservazione. La sottoscrizione si annulla con `Dispose()`.

19. Interrogare sequenze osservabili con operatori LINQ (sl. 35-36)

- **Concat** (combinazione di stream): `source1.Concat(source2)` con `Range(1,3)` produce 1,2,3,1,2,3. La seconda sequenza non è attiva finché la prima non ha finito di spingere tutti i suoi valori; solo dopo il completamento di `source1` `source2` inizia a spingere.
- **Merge**: `source1.Merge(source2)` produce 1,1,2,2,3,3. Le due sequenze sono attive contemporaneamente e i valori sono spinti man mano che si verificano nelle sorgenti; la sequenza risultante si completa solo quando l'ultima sorgente ha finito.
- **Proiezione**: da un `Observable.Range(1, 5)` con `from n in seqNum select new string('*', (int)n)` si mappa uno stream di numeri in uno stream di stringhe.
- Altri operatori importanti: filtering, operazioni temporali, gestione delle eccezioni. In generale: **trasformare** i dati (aggregazione, proiezione), **comporre** i dati (zip, selectMany), **interrogare** i dati (where, any).

20. Riferimenti del modulo 2.2 (sl. 37)

- Lee e Messerschmitt 1987 (Synchronous Data Flow); Elliott e Hudak 1997 (Functional reactive animation); Cooper e Krishnamurthi 2006 (Embedding dynamic dataflow in a call-by-value language); Edwards 2009 (Coherent reaction); Meijer 2012 (Your mouse is a database); Bainomugisha et al. 2013 (A Survey on Reactive Programming); Salvaneschi et al. 2014 (Towards Distributed Reactive Programming, OOPSLA); Mogk et al. 2018 (Fault-tolerant Distributed Reactive Programming, ECOOP).